Conference Defence Q&A Handbook

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Debopam Sinha

How to answer in the conference: Answer directly in two or three sentences. Do not over-explain unless the professor asks a follow-up. For a limitation, acknowledge it confidently and state the next experiment that would address it.

1. Basic Concepts

Q: What is a Hardware Trojan?
Answer: A Hardware Trojan is an intentional malicious modification of a hardware design. It can remain dormant during normal operation and activate only under a particular condition to alter functionality, leak information, degrade performance, or create another security risk.

Q: What is RTL?
Answer: RTL stands for Register Transfer Level. It is a hardware-design abstraction that describes how data moves between registers and how combinational logic processes that data, usually using Verilog or VHDL.

Q: Why is RTL-level Trojan detection important?
Answer: RTL enables security analysis early in the design flow, before synthesis and physical implementation. Detecting suspicious logic at this stage can reduce the cost and difficulty of correcting a design later.

Q: What is the difference between RTL, gate level, and physical design?
Answer: RTL describes intended functionality using constructs such as registers, conditions, and assignments. Gate level represents the synthesized netlist of logic gates. Physical design represents placement and routing on silicon. Our work evaluates only RTL-level analysis.

Q: What are a trigger and a payload in a Hardware Trojan?
Answer: The trigger is the condition that activates the Trojan, often a rare combination of inputs or internal states. The payload is the malicious behavior that occurs after activation.

Q: Can you give a simple trigger-payload example from your work?
Answer: In the illustrative counter example, the trigger becomes active when `en = 1` and the counter reaches decimal value 10. The payload then affects the counter behavior through an additional signal.

Q: Why can Hardware Trojans escape normal verification?
Answer: Verification usually tests expected operating conditions and practical test scenarios. A Trojan can use a rare trigger condition that is not exercised during normal simulation or testing.

Q: What is the difference between a bug and a Hardware Trojan?
Answer: A bug is generally an unintended design error. A Hardware Trojan is intentionally inserted malicious logic. Their observable behavior can sometimes look similar, but their origin and threat model differ.

Q: Why use Verilog in this work?
Answer: Verilog is widely used for RTL design and is a structured hardware description language. Its code-like form also makes it suitable for transformer-based language-model processing.

Q: What does Trojan detection mean in your paper?
Answer: Trojan detection is binary classification of an RTL sample as clean or Trojan-inserted. Our model predicts whether a given serialized RTL prompt contains suspicious Trojan logic.

Q: What does Trojan localization mean in your paper?
Answer: Localization means identifying RTL regions associated with the Trojan trigger and payload. We compare predicted regions against ground-truth metadata recorded during controlled Trojan generation.

2. Motivation and Research Contribution

Q: What problem does your research solve?
Answer: The work studies whether an LLM can perform RTL Trojan detection and localization with a resource-efficient adaptation method. The aim is reproducible academic experimentation without requiring large-scale GPU infrastructure.

Q: What is the main contribution of this work?
Answer: The main contribution is an end-to-end controlled RTL Trojan analysis workflow that combines synthetic Trojan generation, prompt sanitization, LoRA-based adaptation, detection, and localization-aware evaluation in a low-resource setting.

Q: What is novel in your work?
Answer: The novelty is primarily in the engineering integration of a controlled RTL Trojan dataset, label-sanitized prompt preparation, LoRA adaptation of Qwen2.5-1.5B-Instruct, and trigger-payload localization evaluation under constrained hardware. I do not claim a fundamentally new transformer architecture.

Q: Why did you choose an LLM instead of a traditional machine-learning model?
Answer: RTL is structured sequential code with relationships across declarations, conditions, and assignments. A code-capable transformer can learn contextual patterns in serialized RTL, while conventional handcrafted-feature methods may require manual feature design.

Q: Does an LLM understand hardware in the same way as an engineer?
Answer: No. The LLM learns statistical and structural patterns from the representation and training data. It can support security analysis, but its outputs require careful evaluation and should not replace formal hardware-security verification.

Q: Why not use only formal verification?
Answer: Formal verification is powerful and provides stronger guarantees for specified properties, but it can be computationally expensive and requires suitable properties or threat assumptions. Our approach is complementary: it is a learned screening and localization workflow, not a replacement for formal verification.

Q: Why not use side-channel Trojan detection?
Answer: Side-channel methods often require fabricated hardware or measured signals such as power, timing, or electromagnetic behavior. RTL analysis operates earlier in the design process and does not require a physical chip.

Q: Why is resource efficiency important?
Answer: Many academic researchers do not have access to dedicated high-memory GPUs or multi-GPU servers. Parameter-efficient adaptation lowers the number of trainable parameters and makes experimental work more reproducible in such environments.

3. Dataset and Trojan Generation

Q: Where did the clean RTL data come from?
Answer: The clean Verilog RTL samples were collected from the RTL-Coder repository. They were used as the base designs for controlled Trojan generation.

Q: How many samples are in the dataset?
Answer: The final balanced dataset contains 600 RTL samples: 300 clean samples and 300 Trojan-inserted samples.

Q: How did you create Trojan samples?
Answer: We inserted controlled trigger and payload logic into clean Verilog RTL modules while aiming to preserve valid RTL functionality. The insertion process also recorded trigger and payload regions as metadata.

Q: Why use synthetic Trojans instead of real-world Trojans?

Answer: Synthetic generation provides known ground truth for both detection and localization, which supports controlled and reproducible evaluation. However, it is also a major limitation because synthetic Trojans do not cover the full diversity of real-world attack strategies.

Q: Does your dataset contain real industrial Hardware Trojans?
Answer: No. The current work uses controlled synthetic RTL Trojan samples. Testing on heterogeneous real-world benchmarks is an important future direction.

Q: How is the dataset split?
Answer: We used a randomized stratified split. Training contains 420 samples, validation contains 90 samples, and testing contains 90 samples. Each split has equal clean and Trojan class counts.

Q: Why did you use stratified splitting?
Answer: Stratification preserves the clean-to-Trojan class balance in each split. This prevents class imbalance from artificially affecting the classification metrics.

Q: Is the test set balanced?
Answer: Yes. The test set contains 45 clean RTL samples and 45 Trojan-inserted RTL samples.

Q: Could similar RTL templates appear in both training and test sets?
Answer: Yes, that possibility cannot be ruled out with a randomized stratified split. The paper explicitly acknowledges that template-level separation was not enforced, so the results should be interpreted as performance in a controlled setting rather than proof of broad structural generalization.

Q: How would you improve the split in future work?
Answer: I would use template-level, design-family-level, or project-level separation. This would ensure that structurally similar designs do not appear across training and test partitions.

Q: How did you ensure that the Trojan insertion did not break RTL functionality?
Answer: The framework uses controlled, syntactically valid insertion patterns intended to preserve valid RTL behavior. However, the paper does not claim exhaustive formal equivalence checking for every sample, so a stronger future workflow should add compilation, simulation, and formal validation after every insertion.

Q: Could your synthetic generation process create artifacts that make classification easier?
Answer: Yes, that is a valid concern. We reduced obvious artifacts through normalization, lightweight structural obfuscation, and removal of explicit Trojan markers. But synthetic-generation bias can still remain, which is why broader benchmarks and stronger split protocols are necessary.

4. Prompt Preparation and Leakage

Q: What is prompt preparation in your work?
Answer: Prompt preparation converts serialized RTL into an instruction-oriented representation suitable for the LLM. It includes formatting normalization, whitespace cleanup, and removal of explicit annotations that could leak labels.

Q: What are `HT_START` and `HT_END` markers?
Answer: They are temporary markers used during the controlled generation process to identify inserted Trojan regions. They are not retained in the final prompts used by the model.

Q: Why remove Trojan-specific comments and markers?
Answer: If explicit markers remained, the model could learn to classify the label from those markers instead of reasoning from RTL structure. Removing them reduces this obvious shortcut-learning risk.

Q: Can you call the evaluation completely leakage-free?
Answer: I would use the more precise phrase *label-sanitized evaluation*. Explicit labels, markers, and Trojan comments were removed from final prompts, but randomized splitting does not completely eliminate possible structural similarity between training and test samples.

Q: Were ground-truth localization labels given to the model during inference?

Answer: No. The trigger and payload metadata were retained separately for evaluation. During inference, the model received prompt information without disclosure of the ground-truth localization annotations.

Q: How could leakage still occur even after marker removal?
Answer: Leakage can occur through repeated insertion templates, characteristic coding styles, similar RTL designs across splits, or other systematic artifacts. Marker removal addresses only explicit annotation leakage.

Q: Why use obfuscation-aware preprocessing?
Answer: The purpose is to reduce dependence on superficial naming patterns or fixed textual artifacts. The goal is to encourage the model to rely more on RTL structural relationships.

Q: Did you prove that the model learned true hardware semantics?
Answer: No. The reported results show strong performance within the controlled evaluation setup, but they do not prove full semantic hardware understanding. Additional out-of-distribution and template-separated experiments would be needed.

5. LLM, LoRA, and Training

Q: What is Qwen2.5-1.5B-Instruct?
Answer: It is an instruction-tuned large language model with approximately 1.5 billion parameters. We selected it because it is moderately sized and supports instruction-oriented learning workflows.

Q: Why did you choose Qwen2.5-1.5B-Instruct?
Answer: It provides a practical balance between model capacity and resource requirements. A much larger model would increase memory and training cost, while a much smaller model might have less capacity for RTL pattern learning.

Q: What is fine-tuning?
Answer: Fine-tuning adapts a pretrained model to a specific domain or task using task-specific data. In this work, the target task is RTL Trojan detection and localization.

Q: What is LoRA?
Answer: LoRA stands for Low-Rank Adaptation. Instead of updating all pretrained model weights, it adds small trainable low-rank adapter matrices while keeping most backbone parameters frozen.

Q: Why does LoRA reduce computational cost?
Answer: LoRA updates only a small subset of parameters. This reduces optimizer-state memory, gradient storage, and training computation compared with full fine-tuning.

Q: What does PEFT mean?
Answer: PEFT means Parameter-Efficient Fine-Tuning. LoRA is one widely used PEFT method.

Q: What is the difference between LoRA and full fine-tuning?
Answer: Full fine-tuning updates most or all model parameters. LoRA freezes most pretrained parameters and trains only lightweight adapter parameters.

Q: What is the difference between LoRA and QLoRA?
Answer: LoRA adds low-rank trainable adapters to a model. QLoRA combines this approach with low-bit quantization of the base model. The final workflow reported in this paper used FP16 LoRA adaptation for runtime stability and reproducibility.

Q: What was the LoRA rank in your work?
Answer: The LoRA rank was 8. Rank controls the capacity of the low-rank adapter update. A larger rank can increase adaptation capacity but also increases trainable parameters and resource use.

Q: Which exact transformer layers did you adapt with LoRA?
Answer: The paper reports the LoRA configuration, rank, and trainable parameter count, but it does not provide a complete layer-by-layer target-module specification. I would not claim unreported implementation details. A public reproducibility package should document the exact target modules.

Q: What does FP16 mean?
Answer: FP16 means 16-bit floating-point precision. It reduces memory use and can improve training speed compared with 32-bit floating-point computation, subject to numerical-stability considerations.

Q: What learning rate did you use?
Answer: The learning rate was 2×10^{-4}, also written as `2e-4`.

Q: What were the key training settings?
Answer: The reported configuration used LoRA rank 8, FP16 precision, batch size 1, gradient accumulation 8, three training epochs, and a learning rate of 2×10^{-4}.

Q: Why was batch size set to 1?
Answer: A batch size of 1 helps keep GPU memory use low when adapting an LLM on a Tesla T4. Gradient accumulation of 8 helps approximate a larger effective batch while preserving the memory constraint.

Q: What is gradient accumulation?
Answer: Instead of updating the model after every individual sample, gradients are accumulated over multiple steps before an optimizer update. Here, accumulation of 8 supports training stability while retaining a physical batch size of 1.

Q: How many trainable parameters were used?
Answer: The LoRA configuration used 2.17 million trainable parameters, reported as 0.141 percent of the total model parameters.

Q: Why is the trainable percentage so small?
Answer: Most Qwen backbone weights remain frozen. Only the LoRA adapter parameters are optimized for the RTL Trojan task.

Q: What hardware did you use?
Answer: The workflow ran in Google Colab on an NVIDIA Tesla T4 GPU.

Q: Is a Tesla T4 a consumer GPU?
Answer: Strictly speaking, the Tesla T4 is a data-center GPU. The point of the paper is that it is accessible through common academic cloud environments such as Google Colab, rather than requiring a dedicated high-end multi-GPU server.

Q: What does the reported six-minute runtime include?
Answer: The paper reports an approximate training runtime of six minutes for the stated controlled setup. Runtime can vary with Colab allocation, data preparation, environment initialization, and implementation details, so it should be treated as an approximate reproducibility indicator rather than a universal benchmark.

6. Metrics and Results

Q: What are the test-set results?
Answer: On the 90-sample controlled test set, the framework achieved 98.89% accuracy, 100.00% precision, 97.78% recall, and 98.88% F1-score.

Q: What was the confusion matrix?
Answer: There were 45 clean samples correctly predicted as clean, zero clean samples incorrectly predicted as Trojan, one Trojan sample predicted as clean, and 44 Trojan samples correctly predicted as Trojan.

Q: Why is the accuracy 98.89%?
Answer: There were 89 correct predictions out of 90 test samples. Therefore, accuracy is 89/90, which equals approximately 98.89%.

Q: Why is the precision 100%?
Answer: Precision for the Trojan class is $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$. There were 44 true Trojan predictions and zero false-positive Trojan predictions, so precision is $44/(44+0) = 100\%$.

Q: Why is recall 97.78%?
Answer: Recall for the Trojan class is $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$. There were 44 detected Trojan samples and one missed Trojan sample, so recall is $44/(44+1) = 97.78\%$.

Q: Why is F1-score lower than precision?
Answer: F1-score is the harmonic mean of precision and recall. Although precision is 100%, recall is lower because one Trojan sample was missed.

Q: Which error is more serious for Hardware Trojan detection: false positive or false negative?
Answer: Both matter, but they have different costs. A false negative may allow a Trojan to remain undetected, while a false positive can create unnecessary verification effort. In security-sensitive applications, false negatives are often especially concerning.

Q: What are TLC and PLC?
Answer: TLC is Trigger Localization Coverage, and PLC is Payload Localization Coverage. They measure how much of the ground-truth trigger or payload region is covered by the predicted RTL region.

Q: What is the TLC formula?
Answer: The paper uses:

$$\mathrm{TLC} = \frac{|T_p \cap T_g|}{|T_g|}$$

where T_p is the predicted trigger-node set and T_g is the ground-truth trigger-node set.

Q: What is the PLC formula?
Answer: The paper uses:

$$\mathrm{PLC} = \frac{|P_p \cap P_g|}{|P_g|}$$

where P_p is the predicted payload-node set and P_g is the ground-truth payload-node set.

Q: Why are TLC and PLC both 98.89%?
Answer: They are the reported trigger and payload coverage results under this controlled evaluation. Since these are node-level coverage measures, the percentage should not automatically be interpreted as one missed sample out of 90.

Q: Does high TLC or PLC prove exact localization?
Answer: No. TLC and PLC are coverage-oriented measures. They show how much of the ground-truth region the prediction covers, but they do not by themselves strongly penalize predicting extra irrelevant nodes.

Q: What additional localization metrics would strengthen the work?
Answer: Localization precision, intersection-over-union, exact-match rate, and region-level F1-score would provide a more complete assessment. They would quantify both coverage and unnecessary predicted regions.

Q: Why did you not report confidence intervals?
Answer: The current paper reports point estimates on a 90-sample test set. Confidence intervals, repeated runs with different seeds, and statistical comparisons would strengthen future evaluation, especially because the test set is modest in size.

Q: Can you claim 98.89% accuracy in all real RTL designs?
Answer: No. The correct claim is that the framework achieved 98.89% accuracy on the controlled randomized test set used in this study. It should not be generalized to all real-world RTL or industrial hardware without broader validation.

7. Comparison, Validity, and Limitations

Q: How does your work compare with PEARL?
Answer: PEARL provides important context for LLM-based Hardware Trojan analysis. However, it uses a

different model, adaptation strategy, evaluation setting, and hardware environment. Therefore, our paper treats it as a contextual reference rather than a directly comparable benchmark.

Q: Why did you not directly compare accuracy with PEARL?
Answer: A direct comparison would not be scientifically fair without using the same dataset, split, threat model, prompts, task definition, and evaluation protocol. Different experimental settings can make numerical comparisons misleading.

Q: Does your work outperform PEARL?
Answer: I would not make that claim. The focus of our work is resource-efficient LoRA-based adaptation under a controlled setup, not direct performance competition against PEARL.

Q: Did you compare against a classical machine-learning baseline?
Answer: The accepted paper focuses on establishing the controlled LoRA-based prototype and does not report a matched classical baseline. Adding baselines such as rule-based detection, graph-based models, or non-adapted LLM prompting would strengthen future work.

Q: What is the biggest limitation of your work?
Answer: The biggest limitation is external validity. The dataset uses controlled synthetic Trojans, and the randomized split does not enforce template-level separation. Therefore, strong results within this setup do not yet establish broad real-world generalization.

Q: Is the paper production-ready for industrial chip security?
Answer: No. It is an engineering-oriented research prototype. Industrial deployment would require evaluation on real-world benchmarks, gate-level and physical-design validation, robust threat modeling, formal checks, and integration with established verification workflows.

Q: Can an attacker evade your detector?
Answer: Potentially, yes. An attacker may use a structurally different trigger, stealthier payload, adversarial RTL formatting, or logic that differs from the synthetic templates. Robustness against adaptive adversaries requires dedicated security evaluation.

Q: Could the model hallucinate a suspicious RTL region?
Answer: Yes. LLM outputs can be imperfect or overconfident. Localization should therefore guide human review or downstream verification rather than act as an unquestioned security verdict.

Q: Does your method prove that an RTL design is Trojan-free?
Answer: No. A classifier cannot provide a formal proof of Trojan absence. The model can prioritize suspicious designs or regions for further investigation, but formal verification and expert analysis remain necessary.

Q: Can this approach detect a Trojan introduced after synthesis?
Answer: Not in its current form. The present work is limited to RTL. Gate-level netlists and post-synthesis modifications require separate representations, datasets, and evaluation methods.

Q: Can this approach detect analog or physical Trojans?
Answer: No. The current approach analyzes serialized RTL and does not model analog effects, layout-level manipulations, doping-level Trojans, or supply-chain physical attacks.

Q: Why should reviewers trust the result despite these limitations?
Answer: The results are useful because the dataset construction, split, configuration, and limitations are stated clearly. The paper presents evidence for a controlled low-resource prototype, while avoiding the unsupported claim that it solves all Hardware Trojan detection problems.

8. Tough Professor Questions

Q: Your accuracy is very high. Why should I believe it is not caused by dataset artifacts?
Answer: That concern is valid. We removed explicit markers and Trojan-specific comments, and used preprocessing to reduce superficial shortcuts. However, synthetic-template artifacts and structural simi-

larity may still remain, so the results should be interpreted as controlled-setting performance. Template-separated and external-benchmark evaluation are the necessary next steps.

Q: If the dataset is synthetic, are you detecting Trojans or merely detecting your own insertion pattern?
Answer: The controlled setup evaluates whether the model can detect and localize the inserted RTL patterns after explicit label artifacts are removed. It does not prove performance against every attacker-designed Trojan. That distinction is exactly why the paper frames the work as a reproducible prototype and proposes real-world benchmark evaluation as future work.

Q: Why should a professor accept this as research rather than a simple fine-tuning exercise?
Answer: The value lies in the complete security-analysis workflow: controlled Trojan generation with localization ground truth, sanitization to reduce obvious leakage, parameter-efficient adaptation, and joint detection-localization evaluation under limited hardware. The research contribution is an evaluated systems and methodology contribution, not a claim of inventing LoRA or transformers.

Q: What would happen if you removed LoRA and used the base model without adaptation?
Answer: That would be an important ablation study. The accepted paper does not report that result, so I would not invent a number. A future ablation should compare zero-shot prompting, few-shot prompting, LoRA adaptation, and possibly full fine-tuning under the same split.

Q: What would happen if you changed the LoRA rank from 8?
Answer: The trade-off would likely be between adaptation capacity and resource cost. Lower ranks may reduce trainable parameters but limit adaptation, while higher ranks may improve task fit at increased memory and training cost. This should be tested systematically through an ablation study.

Q: Why only three epochs?
Answer: Three epochs were part of the reported low-resource configuration. More epochs may improve fitting or may increase overfitting, especially with a small controlled dataset. Validation-based model selection and a learning-curve study would make this decision stronger.

Q: Why is the test set only 90 samples?
Answer: The 600-sample balanced dataset was split into 420 training, 90 validation, and 90 testing samples. The test set is sufficient for the reported controlled experiment, but it remains modest. Larger external test sets and repeated trials are necessary for stronger statistical confidence.

Q: Why did you not use k-fold cross-validation?
Answer: The study used a fixed randomized stratified split to maintain a simple and reproducible experimental protocol. Cross-validation, especially grouped cross-validation by design template, would provide a stronger estimate of generalization in future work.

Q: Can TLC and PLC be high even when the model predicts many extra nodes?
Answer: Yes. Because TLC and PLC use ground-truth coverage in the denominator, they mainly measure how much relevant ground truth is recovered. Extra predicted nodes are not fully penalized, so localization precision or IoU should be added in future work.

Q: What is your exact threat model?
Answer: The current threat model assumes controlled RTL-level insertion of trigger and payload logic into clean Verilog modules. It does not claim coverage of all adversarial strategies, post-synthesis attacks, or physical supply-chain attacks.

Q: What should be the next immediate experiment after this paper?
Answer: The most important next experiment is a template-separated evaluation on larger and more diverse RTL benchmarks, combined with matched baselines. This would test whether performance remains strong when structural similarity across training and test data is reduced.

Q: If you had more GPU resources, what would you change?
Answer: I would first increase experimental rigor rather than simply use a larger model. I would run ablations, multiple random seeds, grouped splits, external benchmarks, localization precision metrics, and comparisons with full fine-tuning or other PEFT configurations.

9. Final Defence Statements to Memorize

If asked about the 98.89% result:
"The framework achieved 98.89% accuracy on our controlled 90-sample randomized test set. I present it as a strong result within that experimental setting, not as a universal claim for all real-world RTL Trojans."

If asked about novelty:
"The contribution is an end-to-end, resource-efficient RTL Trojan analysis workflow that combines controlled generation, label-sanitized prompts, LoRA adaptation, and localization-aware evaluation."

If asked about the main limitation:
"The main limitation is that the dataset is synthetic and the randomized split does not explicitly enforce template-level separation. The next step is grouped evaluation on broader and more realistic benchmarks."

If asked whether this replaces formal verification:
"No. This work is best viewed as an intelligent screening and localization aid. Formal verification and expert review remain essential for security-critical hardware assurance."

If you do not know an answer:
"That specific implementation detail is not reported in the accepted paper, so I do not want to make an unsupported claim. Based on the current methodology, the correct next step would be to evaluate it through a controlled ablation or validation experiment."